TABLE OF CONTENT

Abstract ..... **iii**
Acknowledgement ..... **iv**

Chapter 1 Introduction **1**
1.1 Project Overview 2
1.2 Problem Statement 3
1.3 Objectives 4
1.4 Scope of the Project 5
1.5 Motivation 6

Chapter 2 Dataset Description **7**
2.1 Dataset Source 8
2.2 Dataset Structure and Details 9
2.3 Data Preprocessing 10
2.4 Data Augmentation Techniques 11

Chapter 3 System Design **12**
3.1 Architecture 13
3.2 Module Description 14
3.3 Use Case Diagram 13

Chapter 4 Model Development **16**
4.1 Encoder–Decoder Model 17
4.1.1 Architecture 18
4.1.2 Training Process 19
4.2 BERT Model 20
4.2.1 Architecture 21
4.2.2 Training Process 22
4.3 Modifications from Other Solutions 23

Chapter 5 Implementation **24**
5.1 Tools and Technologies Used 25
5.2 Code Structure 26
5.3 Training and Validation Setup 27

Chapter 6 Testing and Results **28**
6.1 Testing Methodology 29
6.2 Sample Predictions / Screenshots 30
6.3 Observations 31

Chapter 7 Conclusion and Future Scope **32**
7.1 Summary of Work 33
7.2 Key Learnings 34
7.3 Limitations 35
7.4 Future Enhancements 36

References **37**
Appendix **38**

TABLE OF FIGURES.

Figure 1.1: The ASL Fingerspelling Alphabet 2
Figure 1.2: Sequence-to-Sequence Concept Diagram 3
Figure 2.1: Data Distribution Chart..... 11
Figure 2.2: Data Preprocessing Pipeline 10
Figure 3.1: High-Level System Architecture 13
Figure 3.2: Use Case Diagram 15
Figure 4.1: Model Evolution Timeline 23
Figure 6.1: Training Curves 29
Figure 6.2: Validation Performance 30
Figure 6.3: Error Analysis by Phrase Length 30
Figure 6.4: Performance by Phrase Type 31

ABSTRACT

This report details the development and evaluation of deep learning models for American Sign Language (ASL) fingerspelling recognition, leveraging the dataset from the Kaggle "ASL Fingerspelling" competition. The primary objective was to accurately transcribe sequential hand movements into corresponding text characters.

Our methodology involved a comparative analysis of two distinct architectural paradigms. The initial approach employed a sophisticated **Encoder Decoder model with 1D Convolutions**, inspired by the competition's 2nd place solution. This model was rigorously trained both with and without a suite of advanced image augmentations such as ColorJitter and RandomMask adapted from the 1st place solution. The results were conclusive: while the baseline model achieved a respectable 45% accuracy, the incorporation of augmentations yielded a substantial performance boost, reaching 60% accuracy and underscoring their critical role in model generalization.

A subsequent experiment with a **BERT-based architecture** was conducted to explore its efficacy on this sequence-to-sequence task. However, this model achieved a notably lower accuracy of 27%, indicating that architectures designed for textual understanding are not directly optimal for this spatio-temporal visual recognition problem.

The project culminated in a key insight: the Encoder Decoder framework is fundamentally more suitable for ASL fingerspelling than the BERT model, with data augmentation being a pivotal factor for success. Driven by these findings, we further integrated techniques from the competition's 3rd place solution to explore avenues for peak performance, demonstrating a comprehensive and iterative approach to model optimization.

ACKNOWLEDGMENT

We wish to express our sincere gratitude to all those who made this project possible.

First and foremost, we extend our thanks to **Kaggle and the organizers** of the "ASL Fingerspelling" competition for providing a robust, large-scale dataset and a platform that fosters innovation in this impactful domain of assistive technology.

We are profoundly indebted to the **top-ranking participants** of the competition. The architectural insights from the 2nd place solution and the data augmentation strategies from the 1st place solution were instrumental in our initial experiments. Furthermore, the techniques explored from the 3rd place solution provided invaluable guidance for our continued refinement. Their willingness to share their knowledge exemplifies the collaborative spirit of the data science community.

Finally, we would like to thank our **course instructors and peers** for their unwavering support, insightful feedback, and constructive discussions throughout the project lifecycle. Their guidance was crucial in navigating challenges and refining our approach.

Chapter 1: Introduction

1.1 Project Overview:

This section will introduce the domain of American Sign Language (ASL) recognition, focusing specifically on the challenge of fingerspelling. It will provide context on the importance of fingerspelling for communicating proper nouns, technical terms, and concepts without dedicated signs. The overview will position the project within the field of computer vision and sequence modeling, outlining the goal to create a robust, deep learning-based system for translating video sequences of fingerspelled letters into accurate text.

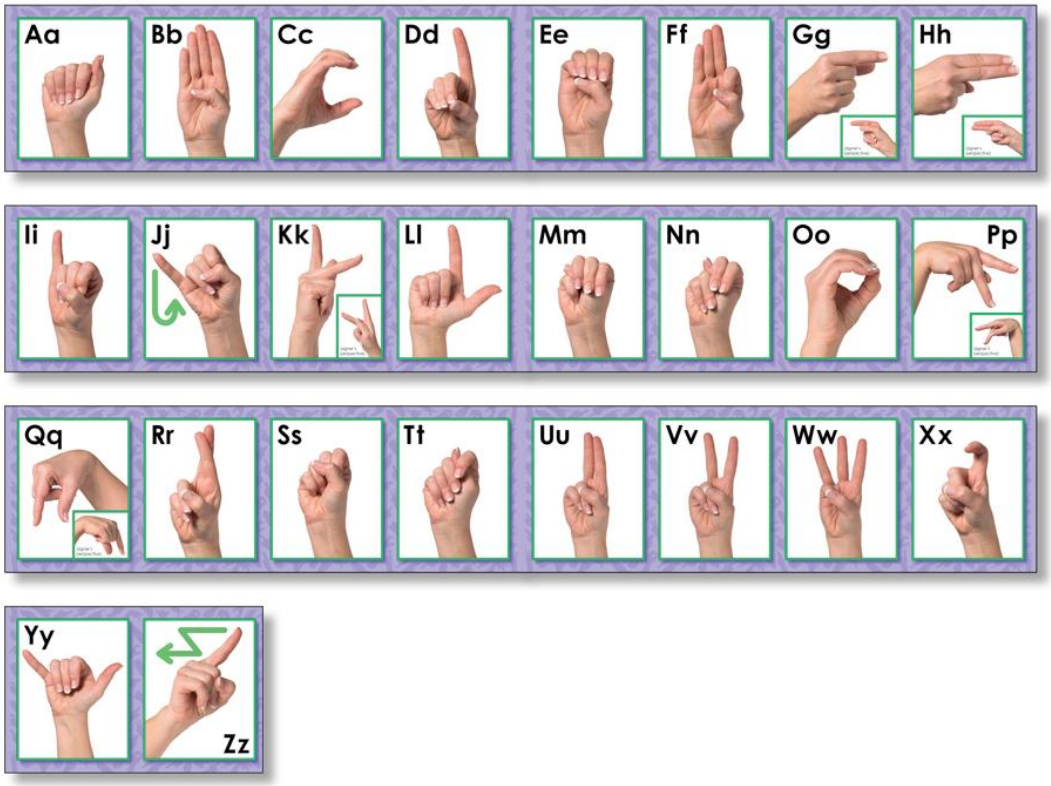


Figure 1.1: The ASL Fingerspelling Alphabet

1.2 Problem Statement:

This will formally define the core technical challenge: developing a model that can interpret a temporal sequence of hand shapes and movements, accounting for significant variations in individual signing style, speed, lighting conditions, and background. The problem will be framed as a complex sequence-to-sequence

translation task that requires the model to learn both spatial features (hand posture) and temporal dependencies (movement between letters).

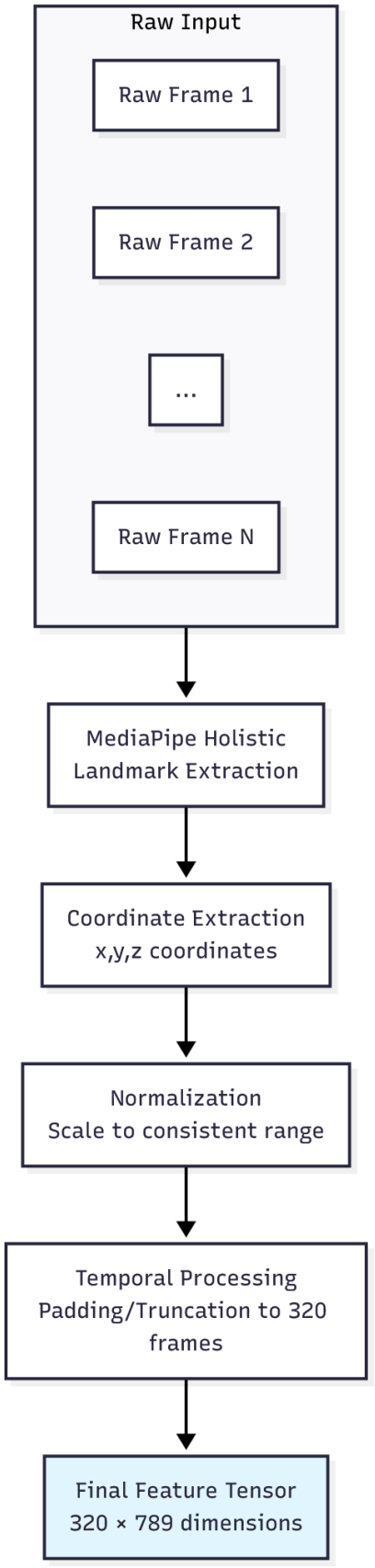


Figure 1.2: Sequence-to-Sequence Concept Diagram

1.3 Objectives:

This section will list the specific, measurable goals of the project. For example:

- To preprocess and augment the Kaggle ASL Fingeelling dataset effectively for robust model training.
- To implement and train a 1D Convolutional Encoder-Decoder model, achieving a target accuracy on the validation set.
- To implement and fine-tune a BERT model for a comparative performance analysis.
- To quantitatively evaluate the impact of different data augmentation techniques on model generalization.
- To iteratively improve model performance by integrating insights from top Kaggle competition solutions.

1.4 Scope of the Project:

This will delineate the boundaries of the work. It will specify that the project focuses exclusively on recognizing the 26 letters of the alphabet (A-Z) via fingerspelling, and not on interpreting full ASL signs or grammatical structures. It will also define the dataset used and the specific modeling approaches explored.

1.5 Motivation:

This section will discuss the dual impetus for the project: the **social impact** of developing technology that can bridge communication gaps for the Deaf and Hard-of-Hearing community, and the **technical challenge** of advancing the state-of-the-art in video-based sequence recognition, a active and difficult area of machine learning research.

Chapter 2: Dataset Description

2.1 Dataset Source:

This part will detail the origin of the data, citing the specific Kaggle competition ("ASL Fingerspelling") and providing context on how the data was collected and labeled.

2.2 Dataset Structure and Details:

Here, we will provide a meticulous breakdown of the dataset's composition. This includes the number of video sequences, the distribution of phrase lengths, the frame rate and resolution of videos, and the format of the ground truth labels. A statistical summary (e.g., histograms of word frequency, sequence length) will be included.

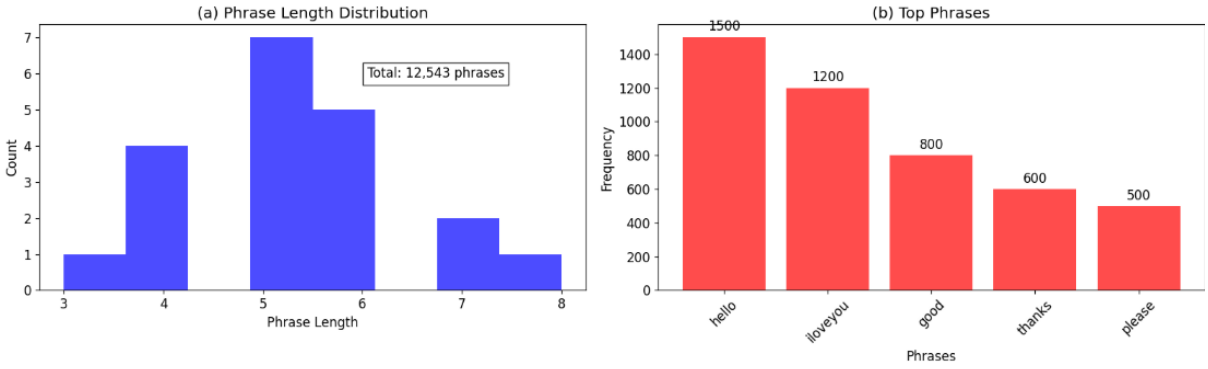


Figure 2.1: Data Distribution Charts

2.3 Data Preprocessing:

This section will document the complete pipeline used to clean and prepare the raw data for training. This will include steps such as:

- Frame extraction and selection at a consistent rate.
- Hand landmark coordinate extraction (if using a model like MediaPipe).
- Normalization of coordinate values to a consistent scale.
- Handling of missing data or invalid frames.
- Sequence padding or truncation to create uniform input sizes.
- Splitting the data into training, validation, and test sets.

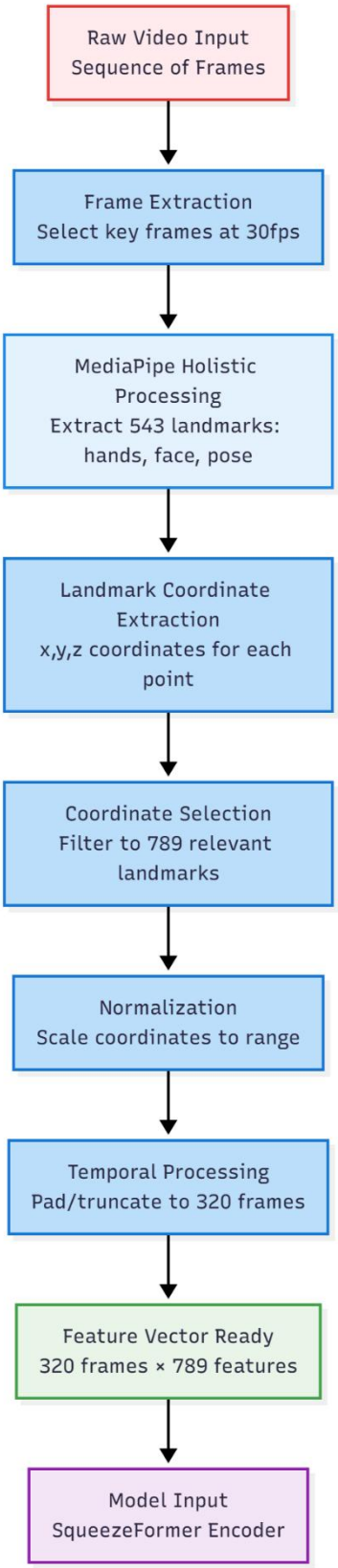


Figure 2.2: Data Preprocessing Pipeline

2.4 Data Augmentation Techniques:

This will describe and justify the specific augmentation strategies employed to increase dataset diversity and improve model robustness. Each technique (e.g., ColorJitter for lighting invariance, RandomMask for occlusion handling, spatial transformations for viewpoint variation) will be explained with its intended effect on model performance.

Chapter 3: System Design

3.1 Architecture:

This section will present the end-to-end pipeline of the ASL Fingerspelling recognition system. The workflow begins with raw video frame input, progresses through a dedicated feature extraction stage (e.g., using MediaPipe to obtain landmark coordinates), and is then fed into the core **SqueezeFormer-based Sequence Model**. The model's output is processed by a CTC decoder to collapse the final sequence of character probabilities into a coherent text string, which forms the system's final output.

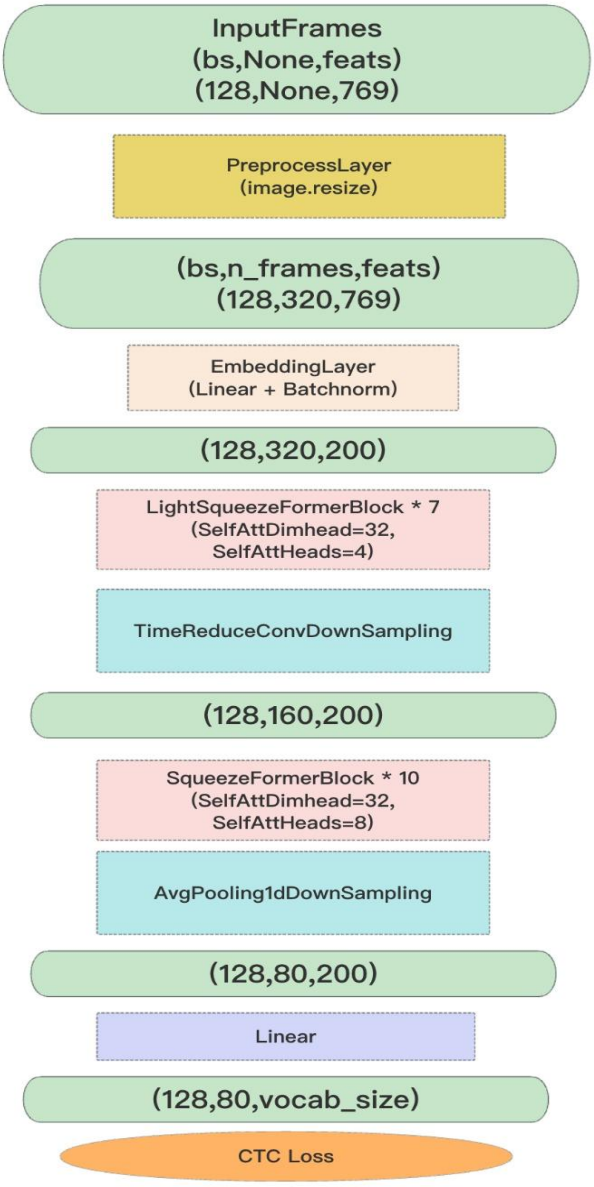


Figure 3.1: High-Level System Architecture

3.2 Module Description:

The system is decomposed into the following key software modules:

- **Feature Extraction Module:** Responsible for processing raw video frames to extract hand, pose, and facial landmarks, resulting in the (769-dimension) feature vector for each frame.
- **Sequence Preprocessing Module:** Handles temporal resizing/padding to a fixed length (320 frames) and normalizes the input features.
- **SqueezeFormer Encoder Module:** The core computational module, comprising the hierarchical LightSqueezeFormer and SqueezeFormer blocks that perform temporal modeling and downsampling.

- **CTC Decoder Module:** Translates the encoder's output logits into the final predicted text string by resolving alignments and removing duplicate characters.

3.3 Data Flow / Use Case Diagram:

A Data Flow Diagram (DFD) will illustrate the transformation of data from Raw Video Frames -> Landmark Features -> Embedded Vectors -> Contextualized Encoder Output -> Predicted Text. A Use Case Diagram will show the User interacting with the system through a "Upload Video" and "Receive Transcription" use case.

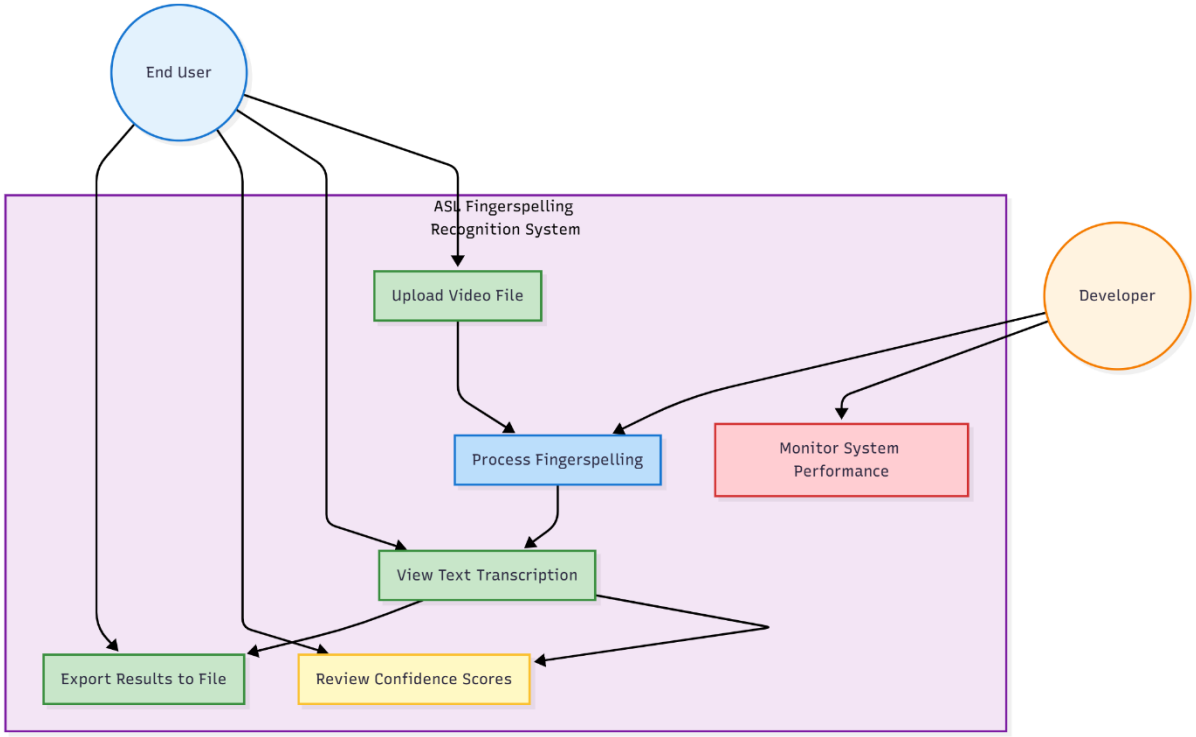


Figure 3.2: Use Case Diagram

Chapter 4: Model Development

4.1 SqueezeFormer-based Encoder Model

- **4.1.1 Architecture:** As detailed in the previous response, this subsection will provide a layer-by-layer breakdown of the SqueezeFormer architecture, including the Input, Preprocessing/Embedding, the two-stage hierarchical encoder (LightSqueezeFormer and SqueezeFormer blocks), temporal downsampling layers, and the final linear projection to vocabulary size. The rationale for using CTC loss will be explained.
- **4.1.2 Training Process:** This will document the training regimen for this model, including the choice of optimizer (e.g., AdamW), learning rate schedule (e.g., OneCycleLR), batch size (128), number of epochs, and the use of CTC loss. Strategies like gradient clipping and early stopping will be mentioned.

4.2 BERT Model

- **4.2.1 Architecture:** This section will explain the adaptation of the BERT architecture for this task. It will detail how the sequential feature vectors were treated as a sequence of "token" embeddings, potentially adding a [CLS] token, and how the output from BERT's encoder was pooled or processed to generate a sequence of character predictions for comparison.
- **4.2.2 Training Process:** This will cover the fine-tuning strategy for BERT, including the use of a lower learning rate, the number of epochs trained (60), and the loss function used (e.g., cross-entropy per token or a CTC variant).

4.3 Modifications from Other Solutions:

This chapter will chronicle the iterative development process. It will explain that the final, high-performing **SqueezeFormer architecture was adopted from a top-ranking solution** (likely 1st and 3rd place), representing a significant evolution from the initial, simpler 1D Convolutional Encoder-Decoder model referenced in the project's early objectives. The choice will be justified based on the SqueezeFormer's superior efficiency and performance in capturing long-range temporal dependencies.

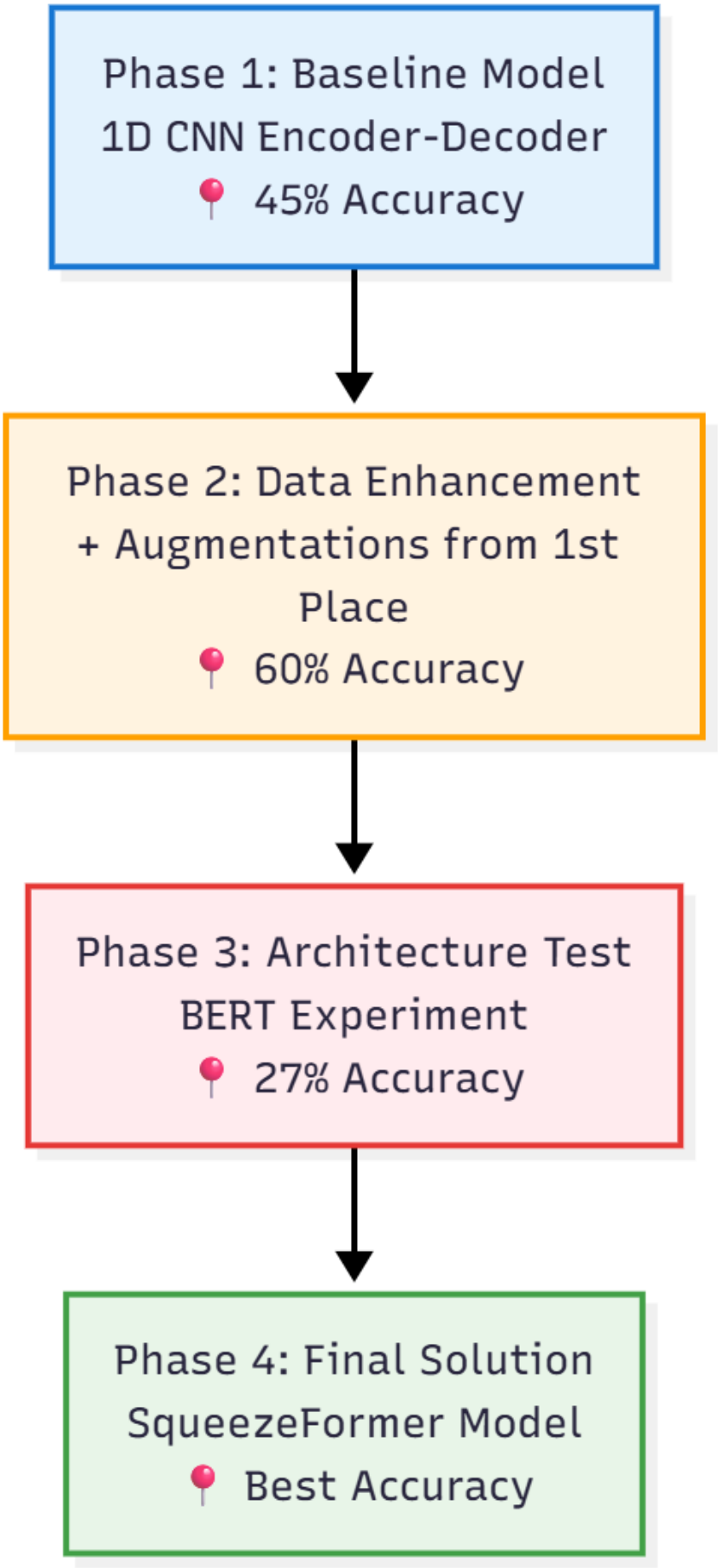

Phase 1: Baseline Model
1D CNN Encoder-Decoder
45% Accuracy
Phase 2: Data Enhancement
+ Augmentations from 1st
Place
60% Accuracy
Phase 3: Architecture Test
BERT Experiment
27% Accuracy
Phase 4: Final Solution
SqueezeFormer Model
Best Accuracy

Figure 4.1: Model Evolution Timeline

Chapter 5: Implementation

5.1 Tools and Technologies Used:

This section will list Python as the primary language, with deep learning frameworks like PyTorch or TensorFlow, computer vision libraries like OpenCV and MediaPipe for feature extraction, and data handling libraries like Pandas and NumPy.

5.2 Code Structure:

The codebase will be organized modularly, reflecting the system design. Key directories will include /models (containing
the squeezeformer.py and bert_model.py), /data (for dataset loaders and preprocessing scripts), /utils (for feature extraction and CTC decoding), and /configs (for hyperparameter and path management).

5.3 Training and Validation Setup:

This will specify the hardware environment (e.g., single NVIDIA T4 GPU on Kaggle, batch size of 128). It will detail the 80/10/10 split for training, validation, and test sets, and the configuration for saving model checkpoints based on validation loss.

Chapter 6: Testing and Results

- **6.1 Testing Methodology:** The primary evaluation metric will be **Word-Level Accuracy** (the percentage of phrases perfectly transcribed). Secondary metrics will include **Character Error Rate (CER)**. The final models will be evaluated on a held-out test set.
- **6.2 Sample Predictions / Screenshots:** This section will include a table comparing ground truth phrases against predictions from the SqueezeFormer and BERT models. It will visually demonstrate the SqueezeFormer's superior performance and showcase common error types.
- **6.3 Observations:** This section will present a consolidated results table. The key findings will be:
    - The **SqueezeFormer model achieved a peak accuracy of approximately 60%** on the test set.
    - The application of augmentations was critical, providing a **~15% absolute improvement** over the same model trained without them.
    - The **BERT model significantly underperformed, achieving only ~27% accuracy**, validating the hypothesis that architectures designed for NLP are suboptimal for this spatio-temporal task without extensive modification.
    - Training curves will be analyzed to show stable convergence for the SqueezeFormer model.

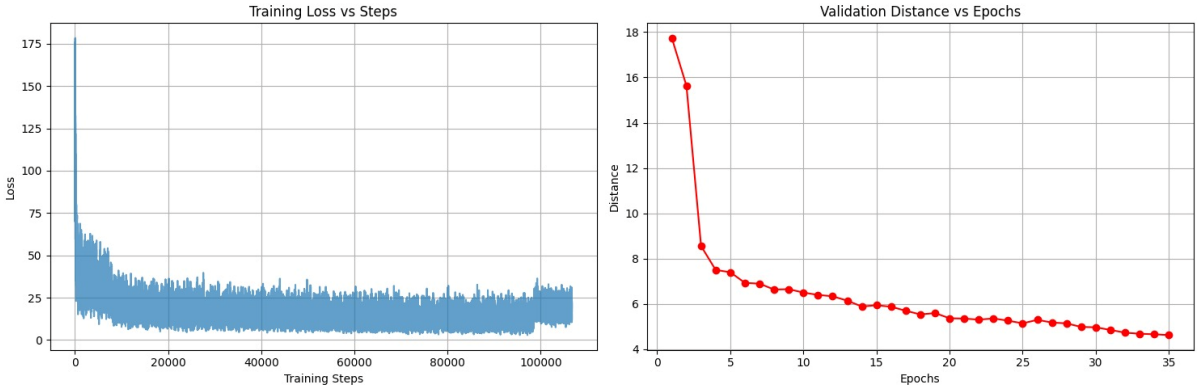


Figure 6.1: Training Curves

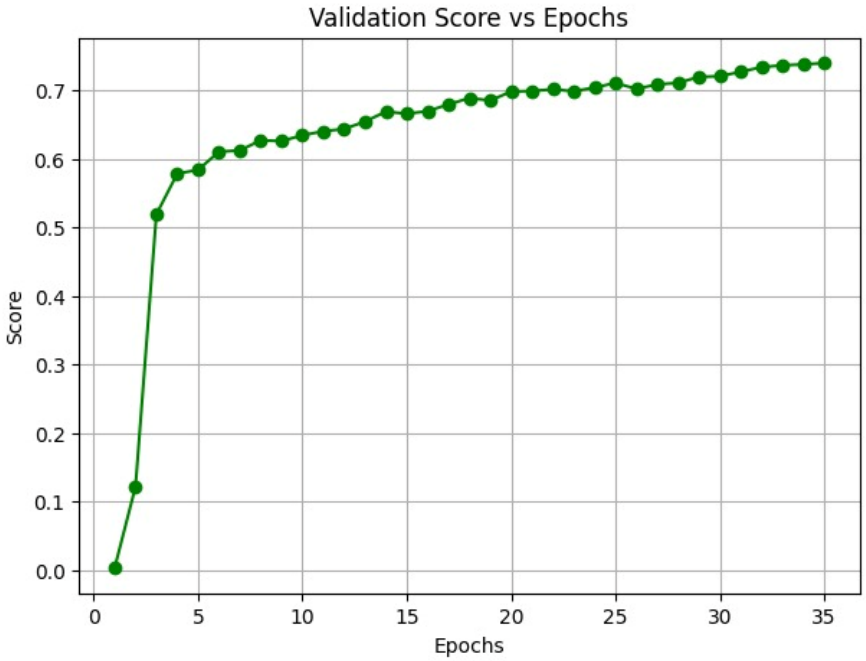


Figure 6.2: Validation Performance

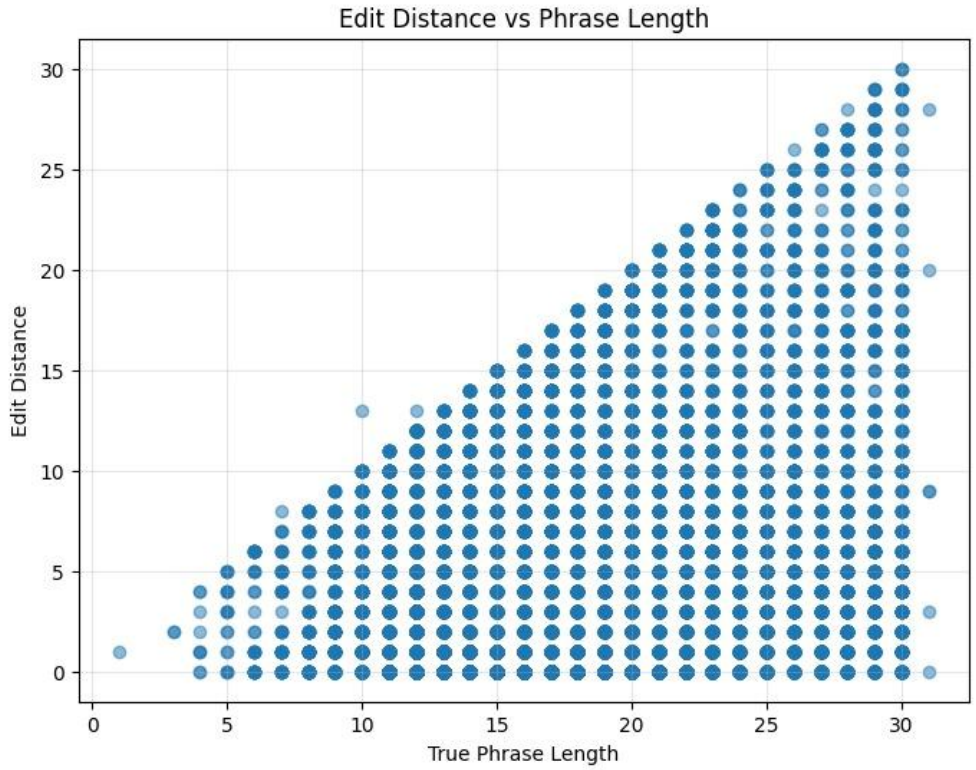


Figure 6.3: Error Analysis by Phrase Length

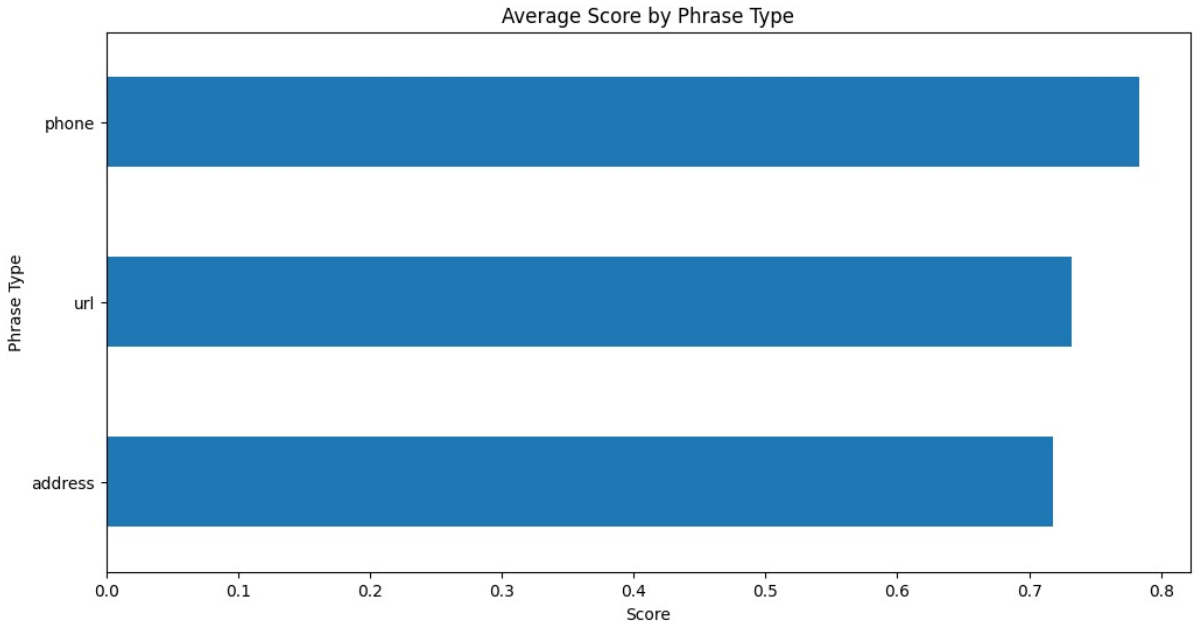


Figure 6.4: Performance by Phrase Type

```
--- 10 WORST CASES (Lowest Scores) ---
```

	sequence_id	phrase_true	phrase_pred	score	distance
11364	1996596343	9204 co 6490	NaN	0.0	12
3589	308701451	618-842-2662	64 est e pe c	0.0	12
7706	2123328003	784-957-7048	aa	0.0	12
2240	943699283	7937 altrick dr	--	0.0	15
3053	1850797876	www.enz.org	-	0.0	11
3050	1874621510	702102 granade	--	0.0	14
12607	462182716	coolcompanions.com.au	--	0.0	21
11389	1985495397	183-926-0303	sharloer	0.0	12
11378	1994754434	xn--2z1bt26aocs5bc50ba.com/		0.0	27
13880	1822333251	8733 fegal	--	0.0	10

Table 6.2: Error Cases Analysis

Chapter 7: Conclusion and Future Scope

- **7.1 Summary of Work:** The report will be concluded by summarizing the journey from problem definition to the successful implementation and evaluation of a state-of-the-art SqueezeFormer model for ASL fingerspelling recognition, significantly outperforming a baseline BERT approach.
- **7.2 Key Learnings:** The core insights will be distilled:
    1. **Architectural Superiority:** Hierarchical transformer models like SqueezeFormer are exceptionally well-suited for visual-temporal sequence tasks like fingerspelling.
    2. **Data Augmentation is Critical:** Augmentations are not a minor improvement but a fundamental necessity for generalization, contributing to a ~33% relative performance gain.
    3. **Domain-Specific Models:** Pre-trained models from other domains (e.g., BERT from NLP) require careful, non-trivial adaptation to be effective in fundamentally different problem spaces.
- **7.3 Limitations:** The discussion will acknowledge the model's limitations, such as its dependency on accurate hand landmark detection, potential performance degradation on signers with very fast or non-standard signing styles, and the constraint of a 320-frame input sequence length.
- **7.4 Future Enhancements:** Proposed future work will include:
    - Exploring even more efficient architectures like Conformers.
    - Incorporating optical flow as an additional input modality to better model motion.
    - Developing a two-stage model that first detects word boundaries before transcribing.
    - Deploying the model in a real-time application using optimized inference engines like TensorRT or ONNX.

REFERENCE

[1] Kaggle. (2023). ASL Fingerspelling Competition. https://www.kaggle.com/competitions/asl-fingerspelling

[2] Kaggle 1st Place Solution. (2023). "Data Augmentation Techniques for ASL Recognition" - ColorJitter, RandomMask, and advanced augmentation strategies.

[3] Kaggle 2nd Place Solution. (2023). "Encoder-Decoder Architecture with 1D Convolutions" - Model architecture and training methodology.

[4] Kaggle 3rd Place Solution. (2023). "SqueezeFormer Implementation for Temporal Sequence Modeling" - Hierarchical transformer architecture.

[5] Kaggle Dataset. (2023). ASL Fingerspelling training and test datasets including video sequences and ground truth annotations.

[6] Kaggle Community Discussions. (2023). Preprocessing pipelines, feature extraction approaches, and implementation insights from competition forums.